

(a) Description

- a. Each message on any exchange between the clients and the server has 4 categories:
 - i. **response status code,**
 - ii. **response command issued,**
 - iii. **response message,**
 - iv. **response data.**
- b. Each of the 4 categories are separated using a delimiter, “.”. Which allows easy extraction of each category.
- c. Each category is a string.
- d. One **response status code** is the string “SUCCESS”, which indicates that a request was fulfilled successfully. The other is the string “FAILURE”, which indicates that something in the request was unacceptable.
- e. The **response command** issued lets both the client and server know the topic of the exchange.
- f. The **response message** gives a readable explanation for what had occurred in the exchange.
- g. The **response data** contains more complex data that would display in a different form. Like the player count with the player list including details about the registered players.
- h. The general format:
 - i. “response_status_code.|response_command.|response_message.|response_data”
- i. The format for each command issued is as follows:
 - i. **register:**
 1. *Client:* “register johndoe”
 2. *Server:* “SUCCESS.|register.|johndoe has been registered!.”
 - ii. **query-players:**
 1. *Client:* “query-players:
 2. *Server:* “SUCCESS.|query-players.|players retrieved!.|{“Num_Of_Players”: 1, “Players”: {“johndoe”: {“status”: “free”, t-port: 1300}}}}”
 - iii. **query-games:**
 1. *Client:* “query-games”:
 2. *Server:* “SUCCESS.|query-games.|games retrieved!.|{“Num_Of_Games”: 1, “Games”: {“2023-1923-1283”: {“Num_Of_Players”: 1, “Players”: {“johndoe”: {“status”: “free”, t-port: 1300}}}}}}”
 - iv. **de-register:**
 1. *Client:* “de-register johndoe”
 2. *Server:* “SUCCESS.|de-register.|johndoe has been de-registered!.”

(b) Time-space diagram

(c) Data-structure and algorithms used

- a. For data structures, I mainly use dictionaries for anything requiring storage. As it makes it easy for me to read when navigating the data.
 - b. The two sets of data that requires a data structures are my player data and game data.
 - c. Player data is a dictionary with 2 values, "Num_Of_Players" and "Players".
 - d. Game data is a dictionary with 2 values, "Num_Of_Games" and "Games".
 - e. Each value is made for readability.
 - f. Printing number of players would be `print(players_Data["Num_Of_Players"])`
 - g. Printing each player would a for player in players loop.
 - h. The "Players" dictionary has keys that are the player names. So accessing a specific player would be `players_Data["Players"]["johndoe"]`.
- (d) GitHub commit history
- a. [Commits · MFourB/SocketProject \(github.com\)](#)
- (e) Video demo link and time stamps
- a. [Socket Project Milestone 1 \(youtube.com\)](#)
 - b. 3(a) **not applicable**
 - c. 3(b) **00:00:08** Compiled programs on two distinct end-hosts.
 - d. 3(c) **00:00:28** Starting the tracker program and starting 3 player process thats each register with the tracker.
 - e. 3(d) **00:01:26** One player issues a query player's command.
 - f. 3(e) **00:02:10** A different player issues a query games command.
 - g. 3(f) **00:03:01** Peers exit with the de-register command.